Week 2 – Getting started with STM32 & RCAP

The purpose of this lab is to help you start software development on the STM Nucleo development board and the RCAP board using STM's standard toolchain. The lab instructions require you to have found your group and have loaned a hardware kit from the Electronics Lab.

1. Laboratory set and group project

During Weeks 2, 3 and 4 you will have an assortment of laboratory exercises to complete. **These will help you complete your first project milestone (MS1) in Week 5**. They are designed for you to go through them in your project pairs.

The set of three prescribed laboratories is designed to assist you by providing instruction and exercises on: (i) a professional development environment and commercial toolchain, (ii) familiarisation with several peripheral modules, such as GPIO, ADC, PWM and UART, and (iii) introduction to the STM32 HAL library APIs and interfacing with the RCAP board to help you write C code to complete the development.

2. Lab objectives

- Use the STM32Cube toolchain to initialise and develop a simple embedded program
- Use the MX configurator and the vendor's HAL APIs to control GPIO pins
- Git setup and recap

3. Hello world

In the embedded world, the simplest program that demonstrates that your MCU and toolchain are functioning as expected is blinking an LED. This is equivalent to a "hello world" program when programming on an OS.

Use the "Introduction to STM32CubeIDE" tutorial from last week to create a ***new*** project for your assignment and configure the GPIO pins for LD1 and LD2. Use a sensible name for your project, such as "StepCounter", as you will use the same project for the rest of the course.

Once the code is generated, open main.c in STM32CubeIDE and add the following two lines of code in the main while loop, just above the comment `/* USER CODE END WHILE */`[1]:

```
HAL_Delay(200);
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

This uses the vendor's hardware abstraction layer (HAL) to toggle the LED pin every 200ms.

Press the build button in the toolbar at the top of the IDE screen, and check that there are no warnings or errors.

Ensure that your board is plugged into the computer and press the debug button . Once the program loads, press the play button to run the program.

Check that the green LED on the Nucleo board, labelled LD1, is blinking.

[1] It is important to only add code in sections labelled as user code. This is because other sections of code are generated by STM32CubeMX and will be overwritten when code is re-generated.

4. Git setup

Now that you have verified your newly developed code works as expected, it is important to save this work as a checkpoint in version control. The following steps help you initialise the project as a git repository, and link it to your assigned project repository on eng-git.

1. In windows explorer, navigate to where you saved your project (not the workspace).
2. Right click on your project folder and select "Git bash here".
3. Make the directory a git repo with a default branch "main" by running the command:
   `git init -b main`
4. Download the recommended `.gitignore` file from learn into your project folder. **Make sure the file starts with a "dot" after it's downloaded. Some browsers remove it.**
5. Check the status of the repository and make sure the files listed in the gitignore file are no longer visible by git:
   `git status`
6. Stage all files for commit (apart from those listed in .gitignore) by running the command:
   `git add --all`
7. Commit the changes with an appropriate commit message by running the command:
   `git commit -m "Initial commit: LD1 blinking"`

At this stage, we cannot push the changes to the remote repository as git does not know where that is. Let's add a link to your group's eng-git repository and call it "origin" per convention.

8. Using a browser, log into eng-git, navigate to your blank group repository and copy the project **HTTPS** URL (not the SSH one):[2]
9. In git bash, specify your username and email address with the following commands:
   `git config --global user.name <username>`
   `git config --global user.email <username>@uclive.ac.nz`
10. Add the remote link, replacing <URL> with the copied URL:
    `git remote add origin <URL>`
11. Push the changes to the remote repository "origin" on branch "main":[3]
    `git push --set-upstream origin main`
12. Refresh your repository page on your browser to confirm the files have been added.

From this point onwards in your project, you should follow the following pattern:

- Make a small change
- Test – verify the change works as expected on your device
- Check what's changed using `git status` and `git diff` (line-by-line changes).
- Stage for commit:
  `git add <files>` for specific files[4], or
  `git add --all` if you are happy with all changes.
- (Optional) discard changes: `git restore <files>`
- Commit staged changes: `git commit -m "Briefly describe change."`
- Push: `git push`

[2] HTTPS is a lot simpler to use. Only use SSH if you know what you are doing.
[3] `--set-upstream` makes a permanent connection between your current local branch (main) and the remote branch (also main). This means that next time you push, you can simply run `git push`.
[4] This is the recommended option – your version control is sacred; you should know every file that goes in.

5. GPIO setup for the RCAP

Before we configure the GPIO pins for the RCAP board, we need to determine how the LEDs and switches are wired to the microcontroller. Refer to the RCAP schematic [2] on Learn to complete the following table:

User label	GPIO pin ID	Input/output	Required pull-up/down	Default (inactive) state
NUCLEO_LD1	PA5	Output	N/A	Low
NUCLEO_LD2	PC9	Output	N/A	High
SW1	PC11	Input	Pull-down	Low
SW2			Pull-down	Low
SW3			Pull-down	Low
SW4			Pull-up	High
JOYSTICK_CLICK			Pull-down	Low
RGB_DS1			N/A	High
RGB_DS2			N/A	High
RGB_DS3			N/A	High
RGB_DS4			N/A	High
RGB_RED			N/A	Low
RGB_GREEN			N/A	Low
RGB_BLUE			N/A	Low

Once the table has been completed, apply these configurations to your project using STM32CubeMX (the visual configurator) and re-generate the code. The default (inactive) state column is primarily there for your reference, but it makes sense to initialise each output GPIO to its inactive state in your configuration using the "GPIO output level" field.

6. Button-controlled LED

Inspect the following code snippet and answer the following questions based on the completed table in section 5 (do not run the code):

```
if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_11))
{
        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_3, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_3, GPIO_PIN_SET);
}
else
{
        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_3, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOD, GPIO_PIN_3, GPIO_PIN_RESET);
}
```

1. Which switch is used to control the LED?
2. Which LED is being controlled?
3. Which colour?
4. While the button is pressed, should the LED be on or off?

Add the code snippet to the user code section of the main while loop and verify your answers. **Do not remove the existing code for blinking LD1.**

Don't forget to commit and push once you've generated working code!

7. Refactor your code

In general, it is important to refactor (tidy up) code as you go to avoid accumulating technical debt.[5] In this case, it is also a good idea to make a clear separation between user code and code generated by STM32CubeMX to minimise the risk of user code being overwritten. We will do this by creating a custom entry-point for user code in a new module "app.c" instead of using the while loop provided.

1. Create a header file in the *Inc* directory and call it "app.h".
2. Declare a function app_main in app.h with the following prototype:

```
void app_main(void);
```

3. Create a new C file in the *Src* directory and call it "app.c".
4. Define the app_main function in app.c to run an infinite loop containing the code you have added so far to the main while loop. Don't forget to include "gpio.h" to be able to use its HAL functions!
5. In main.c, remove the user code from the main while loop and add a call to `app_main()` just above the while loop, under `/* USER CODE BEGIN WHILE */`.
6. Include app.h at the top of main.c directly under `/* USER CODE BEGIN Includes */`.

Test your program carefully before you commit and push your changes.

From now on you can make changes to app.c directly, without having to navigate the generated user code scaffolds.

8. Control the rest of the LEDs

Extend the existing code to do the following. You can use the provided simple RGB driver or write your own. If you use the provided files, you will need to add **rgb.c** to the *Src* subdirectory, and **rgb.h** to the *Inc* subdirectory.

1. The top LED (DS3) illuminates white (all RGB on) while SW1 is pressed, otherwise the LED is off.
2. The bottom LED (DS2) illuminates white (all RGB on) while SW2 is pressed, otherwise the LED is off.
3. The right LED (DS4) illuminates white (all RGB on) while SW3 is pressed, otherwise the LED is off.
4. The left LED (DS1) illuminates white (all RGB on) while SW4 is pressed, otherwise the LED is off.

Don't forget to test, commit and push!

[5] Technical debt is generated when speed of development is prioritised over accuracy, leading to greater challenges in the future. In code, the easiest way to accumulate technical debt is to add layers of hard-to-read code on top of each other.

9. Project cloning instructions

At this point, you may want to get the project cloned on your own PC or your partner may want to have a working copy on their account. Please refer to the cloning instructions on Learn under the *Step Counter Project* page.

10. References

[1] "STM32C071x8/xB Microcontroller Datasheet," ST Microelectronics, 2024.

[2] P. Hof, "ENCE361 Shield Schematics," University of Canterbury, 2024.